Copilot Agent Builder — Agent Inventory & Sharing Governance Export (Microsoft Graph)

Implementation Solution Guide

Version 1.0.0 · 2026-07-24

Contents

Executive Summary 5

The situation . . . 5

The answer . . . 5

What this package delivers . . . 5

Scope and honesty . . . 5

Effort and outcome . . . 6

Solution Overview 6

The question this solves . . . 6

What a "Copilot Agent Builder agent" actually is . . . 6

Where Agent Builder agents surface for an administrator . . . 6

The beta endpoint reality (the direct answer) . . . 7

The supported export approach (what you build) . . . 8

What is honestly available today, and what is not . . . 8

Government cloud . . . 8

Deployment model . . . 9

What is in this solution . . . 9

Requirements 9

R1 — A qualifying administrator role for the Copilot admin surfaces and the Package API . . . 10

R2 — The Graph permission scope `CopilotPackages.Read.All`, with admin consent . . . 10

R3 — (Path B, unattended only) An Entra app registration for app-only runs . . . 11

R4 — The Microsoft Agent 365 entitlement for the Package Management API (VERIFY for your tenant) . . . 11

R5 — Agent Builder enabled and in use in the tenant (the population to inventory) . . . 12

R6 — The Package Management API rolled out to your tenant (Path B; VERIFY) . . . 12

R7 — Cloud instance and endpoint selection (Commercial vs US Government) . . . 12

R8 — A workstation with PowerShell 7+ and the Microsoft Graph SDK (Path B) . . . 13

R9 — A storage location for the exported register and a review owner . . . 13

Global Requirements — Copilot category 13

GC1 — A Microsoft 365 tenant with Microsoft 365 Copilot enabled . . . 14

GC2 — A qualifying administrator role (least privilege) . . . 14

GC3 — Microsoft Graph PowerShell SDK (for the scriptable tasks) . . . 14

GC4 — Awareness that Copilot admin/API features are preview-volatile . . . 15

GC5 — Sovereign-cloud awareness (Commercial vs Government) . . . 15

Implementation 15

Step 0 — Choose your cloud and your path . . . 15

Step 1 — Confirm prerequisites . . . 16
Step 2 — (Path B) Run the preflight and decode the result . . . 16
Step 3 — (Path B) List agents, filtered to Agent Builder only . . . 16
Step 4 — (Path B) Resolve each agent's sharing to users and groups . . . 17
Step 5 — (Path A) Export the inventory from the Microsoft 365 admin center . . . 17
Step 6 — Build the Agent Builder Governance Register . . . 18
Step 7 — Pilot: reconcile one slice before trusting the whole . . . 18
Step 8 — Expand and record the governance decisions . . . 18
Testing & Validation 19
Gate 1 — Preflight returns a clear, correct verdict (Path B) . . . 19
Gate 2 — The API is reachable OR the fallback is confirmed . . . 19
Gate 3 — Inventory is scoped to Agent Builder only (no Copilot Studio) . . . 19
Gate 4 — Inventory is complete against the admin center . . . 19
Gate 5 — Sharing recipients are accurate (Path B) . . . 20
Gate 6 — Group recipients resolve to real groups (Path B) . . . 20
Gate 7 — Owners are populated and ownerless agents are flagged . . . 20
Gate 8 — The register is safe-to-store and reviewable . . . 20
Gate 9 — The recurring export runs unattended (after Best Practices) . . . 21
Gate 10 — Government cloud behaves as documented (if applicable) . . . 21
Troubleshooting 21
T1 — First, capture the exact status code and the endpoint . . . 21
T2 — `401 Unauthorized`: it is the token, not permissions . . . 21
T3 — `403` decision: is it the SCOPE, the ROLE, the ENTITLEMENT, or the ROLLOUT? . . . 22
T4 — `403` caused by missing scope or consent . . . 22
T5 — `403` caused by an insufficient admin role . . . 23
T6 — `403` caused by the Microsoft Agent 365 entitlement (verify — you ruled this out) . . . 23
T7 — `404` (or backend `403`): the API is not rolled out to your tenant/cloud . . . 23
T8 — `200` but the result is empty or missing agents . . . 24
T9 — `200` on the list, but no per-user sharing appears . . . 24
T10 — Distinguishing an undocumented/wrong endpoint from a real limitation . . . 24
Best Practices 25
Stay read-only for governance reporting . . . 25
Prefer `v1.0` over `beta`, and record the version you used . . . 25
Make the export recurring — schedule the monthly run . . . 25
Triage by sharing scope, not alphabetically . . . 25
Keep the register access-controlled — it names people . . . 25

Distinguish "can reach" from "has used" . . . 26
Keep Agent Builder and Copilot Studio governance separate . . . 26
Least privilege and time-bound admin . . . 26
Government cloud: use the supported path and re-test the rest . . . 26
Deliverables Summary 26
What you receive . . . 26
The five PowerShell scripts . . . 27
The artifact you maintain . . . 27
Format . . . 27
Appendix A — PowerShell Scripts 28

Executive Summary

The situation

Employees are creating **Copilot Agent Builder agents** — lightweight declarative agents built inside Microsoft 365 Copilot — and sharing them with colleagues and groups. Governance needs a **complete inventory** of those agents and, for each, a record of **who it is shared with**. An attempt to pull this from Microsoft Graph was **blocked or failed**, leaving an open question: is the endpoint wrong, the permission wrong, the tenant not enabled, or the data simply unavailable?

The answer

The endpoint attempted — `GET /copilot/admin/catalog/packages/{id}`, the **Copilot Package Management API** — is the **correct** one, and its per-agent detail object **does** return the sharing recipients in a property named `allowedUsersAndGroups` (users and groups). The request fails for one of four concrete, distinguishable reasons: a missing Graph permission scope or admin consent, an insufficient admin role, the **Microsoft Agent 365** entitlement the current documentation states for this API, or the API **not yet being rolled out to the tenant**. This package tells the reasons apart and resolves each.

What this package delivers

- A **supported, repeatable export** of every Agent Builder agent — name, owner, status, dates, publish state — with the inventory **filtered to Agent Builder** agents (Copilot Studio explicitly excluded).
- The **granular sharing detail** (which users and groups each agent is shared with), resolved from object IDs to readable names, via the Graph per-agent detail call.
- Two paths to the same result: **Path A** — the Microsoft 365 admin center Agent Registry **Export to CSV **(no code, no extra entitlement);** Path B** — the Graph Package Management API (scriptable, schedulable, includes sharing). Use A when Graph is gated; use B for the recurring, complete register.
- A clear **beta-endpoint diagnostic**: what the endpoint returns, why it blocks/fails, and the exact fix for each `401` / `403` / `404`.
- A maintained `Agent Builder Governance Register` and a **recurring monthly export** so the record stays current.

Scope and honesty

Scope is **Agent Builder (declarative) agents only**; Copilot Studio governance is out of scope. The register records **who an agent can reach** (its allowed audience), not a runtime usage log of who opened it. The **Agent 365 entitlement** and **tenant rollout** for the Graph path are moving targets — the package tells you how to verify your tenant's state rather than asserting one answer, and provides the admin-center fallback so you can produce the inventory today regardless.

Effort and outcome

Delivered as self-guided, click-by-click documentation plus five PowerShell scripts (read-only by default). A competent administrator follows it end to end — no consultant required — and comes away with a trustworthy, repeatable inventory of Agent Builder agents and their sharing, ready for governance review.

Solution Overview

The question this solves

You have people in your organization building **Copilot Agent Builder agents** — the lightweight declarative agents users create inside Microsoft 365 Copilot — and you need two things a governance owner always needs: a **complete inventory** of those agents (name, owner, when it was created and last changed, whether it is published or blocked) and, for each one, **who it is shared with** (which users and which groups). You tried to pull that from Microsoft Graph and the request was **blocked or failed**, so you cannot tell whether the endpoint is wrong, your permissions are wrong, the feature is not switched on in your tenant, or the data simply is not exposed.

This solution answers that question exactly, with a **supported, repeatable export method** you can run on a schedule, and a **clear diagnostic** for why the Graph call fails and what to do about each cause. It is scoped to **Agent Builder (declarative) agents only** — Copilot Studio agents are explicitly out of scope (your Copilot Studio approval gates already govern those) — and it shows you how to **filter the inventory to Agent Builder agents and exclude Copilot Studio** so the register stays clean.

What a "Copilot Agent Builder agent" actually is

Agent Builder is the in-product builder inside Microsoft 365 Copilot (reachable from Copilot Chat → **New agent**) that lets a user create a **declarative agent**: a name, instructions, starter prompts, and optional knowledge sources (SharePoint files/folders, uploaded files, or web content). A declarative agent reuses the Microsoft 365 Copilot orchestrator and models — it *declares* behavior and knowledge rather than bringing its own model. These agents are **built for individuals and small teams** and are shared with a link, not deployed across channels.

They are **not** the same as Copilot Studio agents. Copilot Studio is the standalone low-code maker platform (`copilotstudio.microsoft.com`) that can build declarative agents *and* custom-engine agents, publish them to many channels, and is governed through the Power Platform admin center. Both agent types surface through some of the same admin and Graph surfaces, which is exactly why an inventory has to be **filtered by the platform that built the agent** — otherwise you mix the two populations and the register no longer means what you think it means.

Where Agent Builder agents surface for an administrator

There are two administrator-facing surfaces, and this solution uses both:

1. **Microsoft 365 admin center** → **Agents.** The **Agent Registry** (Microsoft 365 admin center >

Agents > **All agents** > **Registry**) is the central, no-code inventory of every agent available to your organization — Microsoft agents, external partner agents, agents *Published by your org*, and **agents *Shared by creator*** (the Agent Builder agents users have shared). It has an **Export** action that writes a CSV with 30-plus fields per agent (name, status, owner, publisher, platform, created/modified dates, and more). This is the **supported, no-extra-license** inventory path.

1. **Microsoft Graph — the Copilot Package Management API.** `GET /copilot/admin/catalog/packages`

(list) and `GET /copilot/admin/catalog/packages/{id}` (per-agent detail) — the same API the customer tried. The **per-agent detail** call is the one that returns the granular `allowedUsersAndGroups` collection: the exact users and groups an agent is shared with. This is the programmatic, schedulable path, and it is the authoritative source for the **shared-with** detail.

The beta endpoint reality (the direct answer)

The endpoint you tried — `GET /beta/copilot/admin/catalog/packages/{id}` — is **real, documented, and the correct one** for per-agent detail, including sharing. It is the **Copilot Package Management API**. The following facts are current as of this solution's build date and were verified against Microsoft Learn; because the API moved from preview to release very recently, **treat version- and tenant-rollout-specific behavior as volatile and verify against current Microsoft documentation**.

- **The namespace is correct.** The path is `/copilot/admin/catalog/packages` and

`/copilot/admin/catalog/packages/{id}` — not `/admin/...` or `/solutions/...`. You had the right URL.

- **It is no longer beta-only.** Microsoft Learn now documents the API in **both** `v1.0` **and** `beta`.

Prefer `https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages/{id}` for production; the `beta` path still works and returns the same shape but carries the standard "subject to change" caveat. *(Verify against current Microsoft documentation — this GA'd recently and behavior is still settling.)*

- **The detail object DOES return sharing.** `GET .../packages/{id}` returns a `copilotPackageDetail`

object whose `allowedUsersAndGroups` property is a collection of `{ resourceId, resourceType }` entries where `resourceType` is `user` or `group`. That is precisely the "who is this shared with" data you were looking for. The object also returns `acquireUsersAndGroups`, plus summary fields `availableTo` and `deployedTo` (each `all` / `some` / `none`).

- **The list call does NOT return the granular sharing list.** `GET .../packages` returns the summary

`copilotPackage` objects — it tells you `availableTo`/`deployedTo` as `all`/`some`/`none` but **not** which users/groups. To get the recipients you must call `GET .../packages/{id}` **per agent**. That two-step (list → per-id) pattern is the core of the export method.

- **Why your call was blocked or failed** is almost always one of four things, and Troubleshooting

gives you a decision tree that tells them apart: (a) the token is **missing the** `CopilotPackages.Read.All` **scope** or admin consent; (b) the signed-in identity **lacks a qualifying admin role**; (c) the **Microsoft Agent 365 licensing/entitlement** prerequisite the current docs state for this API is not met; or (d) the **API is not yet rolled out/enabled for your tenant** (it released tenant-by-tenant). A `403` points at (a)/(b)/(c); a `404` points at (d) or a wrong URL; a `401` is a token problem.

> ***On Agent 365 (you told us you already ruled this out).*** *We acknowledge that. Note honestly, though, that the* ***current*** *Microsoft Learn pages for both the list and detail calls state: *"Access to the Package Management API requires a Microsoft Agent 365 license."* Earlier preview guidance implied a Copilot admin feature flag was enough. Because this changed as the API released, we treat the*

*entitlement as a **must-verify** item in Requirements rather than asserting a single answer — and we give you a **no-Agent-365 fallback** (the admin center Registry export) so you can produce the inventory today regardless of how that entitlement resolves in your tenant.*

The supported export approach (what you build)

The solution delivers **two coherent paths to the same governance register**, and tells you when to use each:

• **Path A — Microsoft 365 admin center (no code, no extra Graph entitlement).** Open the Agent

Registry, **filter by Platform** to isolate *Microsoft 365 Copilot Agent Builder* agents (excluding Copilot Studio), and **Export to CSV**. This gives you the inventory columns (name, owner, status, dates, platform). It is the fastest way to a defensible inventory and it always works for an administrator.

• **Path B — Microsoft Graph Package Management API (repeatable, scriptable, includes granular

sharing). **Connect with** `CopilotPackages.Read.All`**,** list **packages filtered to the Agent Builder platform and the** `declarativeAgent` **element type, then** call the per-id detail **for each to read** `allowedUsersAndGroups`**, and** resolve **those user/group object IDs to display names. Export to CSV/JSON and schedule it. This is the path that gives you the** shared-with** detail programmatically.

Both paths converge on one artifact you own and maintain: the `Agent Builder Governance Register` — a spreadsheet/CSV with one row per Agent Builder agent, the sharing recipients resolved to names, and review columns (owner-confirmed, sharing-approved, action). Testing proves the register is complete and the sharing is accurate; Best Practices makes the export **recurring**.

What is honestly available today, and what is not

• **Available:** a complete Agent Builder agent inventory (both paths); each agent's owner, status,

created/modified dates, platform, and publish state; and — via Path B's per-id detail — the **users and groups an agent is shared with** (`allowedUsersAndGroups`).

• **Partial / verify:** the **Agent 365 entitlement** requirement for Path B, and the **tenant rollout**

of the API, are both moving targets — Requirements and Troubleshooting tell you how to confirm your tenant's state rather than assuming.

• **Not exposed (be honest):** the Package API surfaces the **allowed** users/groups (the intended

audience), not a runtime log of *who actually opened or ran* the agent — usage/telemetry is a different data source (Copilot usage reporting / audit logs) and is out of scope here. The register answers **"who can this agent reach,"** not **"who used it last Tuesday."**

Government cloud

The Copilot Package Management API (Path B) is documented as available in the **Global** service only — it is **not** available in **US Government L4 (GCC High)** or **US Government L5 (DoD)**, and not in China (21Vianet). For those clouds, **Path A (admin center Registry export)** is the method, subject to Agent Builder itself being available in that cloud — which has parity gaps you must verify. The scripts still accept `-Environment Global|USGov|USGovDoD` and route Graph to `graph.microsoft.us` / `dod-graph.microsoft.us`, but they will report clearly if the Package API is not present in that cloud

rather than fabricating a result. Requirements R7 and Troubleshooting T7 carry the detail. **Do not assume GCC High parity — verify for GCC High.**

Deployment model

The solution follows the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** You confirm prerequisites and pick a path, run a **preflight** that decodes exactly why (or that) Graph works for you, export a **pilot** slice and reconcile it against the admin center, then widen to the whole tenant and schedule the recurring export.

What is in this solution

- **This overview** — what Agent Builder agents are, the surfaces, the beta-endpoint reality, and the

two-path export approach.

- **Requirements** (`02-requirements.md`) — admin roles, exact Graph scopes, the Agent 365 entitlement

reality, and tenant feature prerequisites, each with Description / Why / Verify / Implement-if-missing.

- **Implementation** (`03-implementation.md`) — the ordered build: connect, enumerate, filter to Agent

Builder, resolve sharing, export, schedule.

- **Testing** (`05-testing.md`) — the pass/fail gates that prove the inventory is complete and the

sharing is accurate.

- **Troubleshooting** (`06-troubleshooting.md`) — the blocked/failed-request decision tree.

- **Best practices** (`07-best-practices.md`), **deliverables summary** (`08-deliverables-summary.md`),

and the **executive summary**.

- **Five PowerShell scripts** — preflight/failure-decoder, inventory export, sharing export, Agent

Builder filter, and the governance-register report generator.

Requirements

Each requirement follows the house pattern: **Description / Why required / How to verify / How to implement if missing.** Confirm every item before you run the export — most "blocked or failed" Graph requests trace directly to R1, R2, R3, or R4.

> ***API-version and rollout note (read first).*** *The Copilot Package Management API used in Path B (`/copilot/admin/catalog/packages`) released from preview very recently and is documented in* ***both*** `v1.0` ***and*** `beta`*. Prefer* `v1.0` *in production. Several facts below (the Agent 365 entitlement, the tenant rollout, national-cloud availability) are* ***preview-volatile — verify each against current Microsoft Learn for your tenant*** *rather than assuming. Where a fact is uncertain, this document says so explicitly.*

R1 — A qualifying administrator role for the Copilot admin surfaces and the Package API

• **Description:** The signed-in identity must hold one of the admin roles that Microsoft authorizes for

the Agent Registry and the Copilot Package Management API. Documented qualifying roles include **Global Administrator**, **AI Administrator**, **Cloud Application Administrator**, **Application Administrator**, and **Copilot Administrator**. The Agent Registry documentation states the Graph API "works with the **AI Administrator** role" — **AI Administrator** is the least-privileged practical choice for read-only governance.

• **Why required:** The API authorizes on the caller's role *in addition to* the OAuth scope. A correct

token with an unqualified role returns **403**.

• **How to verify: Microsoft Entra admin center** > **Roles & administrators** > search **AI

Administrator** (or Global Administrator) > confirm your account is listed. After connecting in PowerShell, you can also confirm effective access with the preflight script (R-flow in Implementation Step 2).

• **How to implement if missing:** Assign **AI Administrator** to the governance account:

Roles & administrators > **AI Administrator** > **Add assignments** > select the account > (prefer a time-bound, eligible assignment via **Privileged Identity Management**). Re-sign-in so the new role is in the token.

R2 — The Graph permission scope `CopilotPackages.Read.All`, with admin consent

• **Description:** Path B calls require the delegated (or application) permission `CopilotPackages.Read.All`

(read-only). The read/write scope `CopilotPackages.ReadWrite.All` exists but is **not needed** for inventory and sharing export — stay read-only. Delegated is supported; application permission is also documented as supported for unattended runs.

• **Why required:** Without the scope on the access token (and tenant admin consent granted for it), the

API returns **403** before it ever looks at the data. This is the single most common cause of the customer's failure.

• **How to verify:** Decode the access token (e.g. paste into `jwt.ms`) and confirm `CopilotPackages.Read.All`

is present in the `scp` (delegated) or `roles` (application) claim. The preflight script also reports the scopes on the current context.

• **How to implement if missing:**

• **Interactive/delegated (fastest):** the scripts request the scope at `Connect-MgGraph -Scopes

CopilotPackages.Read.All`; a Global Administrator must **consent** on first use (or pre-consent it).

• **Unattended/application:** register an Entra app (see R3), add `CopilotPackages.Read.All` as an

Application permission under **API permissions** > **Microsoft Graph**, then **Grant admin consent for <tenant>**.

R3 — (Path B, unattended only) An Entra app registration for app-only runs

- **Description:** For a scheduled, unattended export you register an Entra application named

`AGBINV – Agent Governance – Graph Reader` with the **application** permission `CopilotPackages.Read.All` and a certificate credential.

- **Why required:** The recurring monthly export (Best Practices) runs with no signed-in user; app-only

auth with a certificate is the supported unattended pattern. Interactive runs do **not** need this.

- **How to verify: Entra admin center** > **App registrations** > confirm **`AGBINV – Agent

Governance – Graph Reader exists, has CopilotPackages.Read.All (Application) with a green "Granted" consent state, and a valid certificate under Certificates & secrets.

- **How to implement if missing: App registrations** > **New registration** > name it

`AGBINV – Agent Governance – Graph Reader` > **Register**; add the application permission and grant consent (R2); upload a certificate (prefer certificate over client secret). Record the **Application (client) ID** and **Directory (tenant) ID** for the scripts.

R4 — The Microsoft Agent 365 entitlement for the Package Management API (VERIFY for your tenant)

- **Description: Current** Microsoft Learn for both the list and detail calls states: *"Access to

the Package Management API requires a Microsoft Agent 365 license."* Earlier preview guidance had implied only a Copilot admin feature flag was required. Because this changed as the API released, the entitlement is a **must-verify** item, not a settled fact.

- **Why required:** If the entitlement is truly enforced and absent, Path B returns **403** even with a

perfect role and scope — which can look identical to a permissions problem. Distinguishing the two is in Troubleshooting T3.

- **How to verify: Microsoft 365 admin center** > **Billing** > **Licenses** / **Subscriptions** —

look for a **Microsoft Agent 365** (or agent-management) subscription. Then run the **preflight script**: a clean `403` on `/copilot/admin/catalog/packages` *after* R1/R2 are confirmed points at this entitlement (or R6 rollout). *(Verify against current Microsoft documentation — Microsoft may adjust this entitlement as the API matures.)*

- **How to implement if missing:** Two options. **(a)** Acquire the Agent 365 entitlement if your

organization wants the programmatic path. **(b)** Use **Path A** — the Microsoft 365 admin center Agent Registry **Export to CSV** — which produces the inventory **without** the Agent 365 entitlement. You told us you already ruled Agent 365 out; if that is final, **Path A is your supported route to the inventory today**, and Path B remains available for the sharing detail only if/when the entitlement is in place.

R5 — Agent Builder enabled and in use in the tenant (the population to inventory)

• **Description:** Users must actually be creating and sharing Agent Builder agents, and Agent Builder must not be disabled by policy — otherwise the inventory is legitimately empty.

• **Why required:** An empty result is only correct if there are genuinely no Agent Builder agents; you need to know which it is.

• **How to verify: Microsoft 365 admin center** > **Agents** > **All agents** > **Registry** — look for entries under **Shared by creator** with **Platform** = *Microsoft 365 Copilot Agent Builder*. In Copilot Chat, **New agent** should be available to a test user.

• **How to implement if missing:** If Agent Builder is intentionally disabled, that is a valid governance state (record it). If it should be available, review the agent controls in **Microsoft 365 admin center** > **Copilot** / **Integrated apps** agent settings.

R6 — The Package Management API rolled out to your tenant (Path B; VERIFY)

• **Description:** The API released tenant-by-tenant. A tenant that has not yet received it returns **404** (endpoint not present) or **403** (backend feature not enabled) regardless of role/scope.

• **Why required:** This is indistinguishable from a permissions error without a deliberate test — and no tenant-side toggle forces earlier availability.

• **How to verify:** Run the **preflight script**. A `404` on the list endpoint with a valid token and qualifying role indicates the API is not present for the tenant; a `200` confirms it is live.

• **How to implement if missing:** There is no self-service enablement — use **Path A** in the meantime and re-test Path B after Microsoft confirms general availability for your cloud/tenant. *(Verify current rollout state on Microsoft Learn / the Microsoft 365 Message Center — MC1173195 tracks this API.)*

R7 — Cloud instance and endpoint selection (Commercial vs US Government)

• **Description:** Know your cloud and target the right endpoints. Path B's Package Management API is documented as **Global-only** — **not** available in **US Government L4 (GCC High)**, **US Government L5 (DoD)**, or **China (21Vianet)**.

• **Why required:** In a Government cloud, Path B will not return data because the API is not deployed there — not because of a permissions problem. Using the wrong Graph host also fails authentication.

• **How to verify:** Confirm your admin-center URL (`admin.microsoft.com` Commercial vs `portal.office365.us` GCC High) and, after connecting, `(Get-MgContext).Environment`.

• **How to implement if missing / Government substitution table:**

Setting	Commercial (Global)	US Gov GCC High (L4)	US Gov DoD (L5)
Graph host	`graph.microsoft.com`	`graph.microsoft.us`	`dod-graph.microsoft.us`

Setting	Commercial (Global)	US Gov GCC High (L4)	US Gov DoD (L5)
`Connect-MgGraph -Environment`	`Global`	`USGov`	`USGovDoD`
M365 admin center	`admin.microsoft.com`	`portal.office365.us`	`portal.office365.us`
Package Management API (Path B)	**Available**	**Not available — verify for GCC High**	**Not available**
Inventory method	Path A or Path B	**Path A (admin center export)**	**Path A (admin center export)**

In Government clouds, use **Path A**. Also verify that **Agent Builder itself** is available in your Government cloud before expecting any agents to inventory — agent feature parity in GCC High/DoD is incomplete. **Do not assume GCC High parity — verify for GCC High.**

R8 — A workstation with PowerShell 7+ and the Microsoft Graph SDK (Path B)

• **Description:** A machine running **PowerShell 7+** with the **Microsoft.Graph** modules to run the scripts.

• **Why required:** The scripts use `Connect-MgGraph` / `Invoke-MgGraphRequest`.

• **How to verify:** `$PSVersionTable.PSVersion` (≥ 7.0) and `Get-Module Microsoft.Graph -ListAvailable`.

• **How to implement if missing:** Install PowerShell 7+, then `Install-Module Microsoft.Graph -Scope CurrentUser`.

R9 — A storage location for the exported register and a review owner

• **Description:** A controlled folder — `C:\AgentGovernance\Exports` on the run host (or a governed SharePoint/Teams location) — to hold the exported CSV/JSON, and a **named owner** who reviews the `Agent Builder Governance Register`.

• **Why required:** The export lists people and groups (personal-adjacent data); it must live somewhere access-controlled and be owned by someone accountable for acting on it.

• **How to verify:** The folder exists and is restricted to the governance team; a named owner is recorded in the register header.

• **How to implement if missing:** Create `C:\AgentGovernance\Exports` (the scripts create it if absent) or a permission-scoped SharePoint library; assign the register owner in your governance model.

Global Requirements — Copilot category

These requirements are common to **every** solution in the **Copilot** category. They sit once at the category level; each solution's own `02-requirements.md` adds the requirements specific to that solution. Each item follows the house pattern: **Description / Why required / How to verify / How to implement if missing.**

GC1 — A Microsoft 365 tenant with Microsoft 365 Copilot enabled

• **Description:** An active Microsoft 365 tenant in which Microsoft 365 Copilot (and, where relevant, the agent surfaces) is turned on.

• **Why required:** Agents, Copilot admin surfaces, and the Copilot Graph APIs only exist in a tenant where Copilot is provisioned.

• **How to verify:** Sign in to the **Microsoft 365 admin center** (`admin.microsoft.com`) and confirm **Copilot** appears in the left navigation, and **Billing** > **Licenses** shows a Microsoft 365 Copilot subscription.

• **How to implement if missing:** Acquire and assign Microsoft 365 Copilot licensing before proceeding; Copilot governance tasks have no meaning until Copilot is present.

GC2 — A qualifying administrator role (least privilege)

• **Description:** The person performing Copilot governance work holds an appropriate Entra admin role —
and no more than needed.

• **Why required:** Copilot admin surfaces and APIs are gated to specific roles; over-privileged accounts violate least privilege and audit expectations.

• **How to verify: Microsoft Entra admin center** (`entra.microsoft.com`) > **Roles & administrators** > confirm the account's assigned roles.

• **How to implement if missing:** Assign the least-privileged role the task requires (this solution names the exact roles in R1) via **Roles & administrators** > (role) > **Add assignments**, ideally time-bound through Privileged Identity Management.

GC3 — Microsoft Graph PowerShell SDK (for the scriptable tasks)

• **Description:** The **Microsoft.Graph** PowerShell modules installed on the workstation used to run any Graph-based script.

• **Why required:** Every script in the Copilot solutions authenticates with `Connect-MgGraph` and calls Graph through the SDK.

• **How to verify:** In PowerShell 7+, run `Get-Module Microsoft.Graph -ListAvailable`.

• **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser` (and install **PowerShell 7+** first if needed).

GC4 — Awareness that Copilot admin/API features are preview-volatile

• **Description:** Copilot admin controls and Graph endpoints evolve quickly; some are in preview or roll out tenant-by-tenant.

• **Why required:** A step that is correct today can change; assuming stability leads to wrong conclusions when a call fails only because a feature has not reached your tenant.

• **How to verify:** Cross-check any Copilot admin/API step against current Microsoft Learn before relying on it in production; note the API version (`v1.0` vs `beta`).

• **How to implement if missing:** Treat preview-flagged behavior as "verify against current Microsoft documentation" and prefer `v1.0` endpoints over `beta` where both exist.

GC5 — Sovereign-cloud awareness (Commercial vs Government)

• **Description:** Knowledge of whether your tenant is Commercial (`graph.microsoft.com`, `admin.microsoft.com`) or a US Government cloud (GCC High `graph.microsoft.us` / `portal.office365.us`; DoD `dod-graph.microsoft.us`), and that Copilot/agent feature parity differs.

• **Why required:** Endpoints and feature availability differ by cloud; several Copilot APIs are Global-only.

• **How to verify:** Confirm the tenant's cloud instance in the admin center URL and in `(Get-MgContext).Environment` after connecting.

• **How to implement if missing:** Select the correct `-Environment` for scripts and consult the per-solution government section for parity caveats.

Implementation

Follow these steps in order. The build produces one artifact — the `Agent Builder Governance Register` — by whichever path your tenant supports. Every object you create is named; reuse the exact name verbatim in Testing, Troubleshooting, and the scripts.

> ***Naming used throughout:*** *Entra app registration* `AGBINV – Agent Governance – Graph Reader`*; export folder* `C:\AgentGovernance\Exports`*; output register* `Agent Builder Governance Register` *(CSV* `AgentBuilder-Governance-Register_<date>.csv`*); scheduled task* `AGBINV – Monthly Agent Inventory Export`*.*

Step 0 — Choose your cloud and your path

1. Confirm your cloud (R7). For Government clouds, the Graph Package Management API is not available —

you will use **Path A**.

1. Choose the path:

• **Path A — Microsoft 365 admin center export** (no code, no Agent 365 entitlement). Use this if Agent 365 is ruled out (your case), if you are in a Government cloud, or if you just want the inventory quickly. Steps 1 and 5–8.

• **Path B — Microsoft Graph Package Management API** (scriptable, includes granular `allowedUsersAndGroups` sharing). Use this if the Agent 365 entitlement and API rollout are in place and you want a recurring, complete register. Steps 1–8.

1. You may run **both**: Path A for the inventory baseline, Path B to enrich it with resolved sharing.

Step 1 — Confirm prerequisites

1. Verify R1 (role — **AI Administrator** is the least-privileged practical choice), R2 (scope `CopilotPackages.Read.All` with consent, Path B), R4 (Agent 365 entitlement — verify), R5 (Agent Builder in use), R7 (cloud), and R8/R9 (workstation, storage).

1. Create the export folder `C:\AgentGovernance\Exports` if it does not exist (the scripts create it automatically, but creating it now lets you set folder permissions first per R9).

Step 2 — (Path B) Run the preflight and decode the result

This is the step that answers "why was my request blocked or failed."

1. On the run host, open **PowerShell 7+**.

2. Run the preflight, targeting your cloud:

```
pwsh -File .\AGBINV-Connect-Preflight.ps1 -Environment Global
```

(Use `-Environment USGov` or `-Environment USGovDoD` for Government clouds.)

1. Read its verdict. The preflight signs in, reports the scopes and roles on your context, then calls `GET /copilot/admin/catalog/packages?$top=1` and **decodes the response**:

• `200 OK` — the API is live for your tenant and your identity is authorized. Proceed to Step 3.

• `401` — token/authentication problem (wrong cloud audience, expired token). Re-connect; confirm `-Environment` matches your tenant.

• `403` — authorized identity but access denied. The preflight prints the likely cause in priority order: (1) missing `CopilotPackages.Read.All` scope / no admin consent → fix R2; (2) role not qualifying → fix R1; (3) Agent 365 entitlement not present → R4; (4) tenant feature not enabled → R6.

• `404` — the API is not present for your tenant/cloud (R6 rollout, or a Government cloud where it is unavailable) → use **Path A**. Troubleshooting T2–T7 expand each branch.

Step 3 — (Path B) List agents, filtered to Agent Builder only

1. Run the inventory export, which lists packages and **filters to Agent Builder declarative agents**,

excluding Copilot Studio:

```
pwsh -File .\AGBINV-Export-AgentInventory.ps1 -Environment Global `
     -OutputPath C:\AgentGovernance\Exports
```

1. Internally this calls

`GET /copilot/admin/catalog/packages?$filter=platform eq 'Microsoft 365 Copilot Agent Builder'` and additionally keeps only packages whose `elementTypes` contains `declarativeAgent`. It writes one row per agent with: `Id`, `DisplayName`, `Type`, `Publisher`, `Platform`, `ElementTypes`, `IsBlocked`, `AvailableTo`, `DeployedTo`, `LastModifiedDateTime`, `Version`.

1. The output file is `AgentBuilder-Inventory_<date>.csv` — the inventory backbone. Note that the

list call gives `AvailableTo`/`DeployedTo` only as `all`/`some`/`none`; the *who* comes in Step 4.

> ***Why the filter matters.*** *Without the `platform` filter you would also capture Copilot Studio agents (`platform` = `Copilot Studio`) and Microsoft/partner agents. The filter is what keeps the register scoped to Agent Builder, so it does not overlap the governance your Copilot Studio approval gates already provide.*

Step 4 — (Path B) Resolve each agent's sharing to users and groups

1. Run the sharing export, which calls the **per-agent detail** endpoint for each agent from Step 3 and

reads `allowedUsersAndGroups`:

```
pwsh -File .\AGBINV-Export-AgentSharing.ps1 -Environment Global `
     -InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv `
     -OutputPath C:\AgentGovernance\Exports
```

1. For each agent it calls `GET /copilot/admin/catalog/packages/{id}`, reads `allowedUsersAndGroups`

(each entry is `{ resourceId, resourceType }` where `resourceType` is `user` or `group`), and **resolves** each `resourceId` to a display name / UPN via `GET /users/{id}` or `GET /groups/{id}`. It also captures `acquireUsersAndGroups`, `availableTo`, `deployedTo`, and `sensitivity`.

1. The output file is `AgentBuilder-Sharing_<date>.csv` — one row per **agent × recipient**, with

`AgentId`, `AgentName`, `RecipientType` (User/Group), `RecipientId`, `RecipientName`, `RecipientUpnOrMail`.

> ***Honest note on the summary fields.*** *If an agent's `availableTo` is `all` (shared with **Anyone in your organization**), there may be no explicit per-user entries to resolve — the register should record that agent as **org-wide shared**, which is itself the highest-attention governance state. The script labels these explicitly rather than showing an empty recipient list.*

Step 5 — (Path A) Export the inventory from the Microsoft 365 admin center

Use this path when Agent 365 is ruled out, in Government clouds, or as an independent cross-check.

1. Sign in to the **Microsoft 365 admin center** (`admin.microsoft.com`; GCC High:

`portal.office365.us`).

1. In the left navigation, select **Agents** > **All agents** > **Registry**.

2. Select **Filter**, and under **Platform** choose **Microsoft 365 Copilot Agent Builder** (deselect Copilot Studio and others). Optionally add **Publisher type** = *Shared by creator* to focus on user-shared agents.

1. Select **Export**. Choose scope **Filtered agents** (so you export only the Agent Builder set you filtered to). Save the CSV into `C:\AgentGovernance\Exports`.

1. The exported CSV includes name, status, channel, created/last-modified dates, publisher, publisher type, version, **owner**, description, platform, and more (30-plus fields). It gives you owner and inventory; it does **not** reliably enumerate the individual shared-with recipients — that granular list comes from Path B's `allowedUsersAndGroups`. Record this limitation in the register.

Step 6 — Build the Agent Builder Governance Register

1. Run the register generator to merge the inventory and sharing exports into the review artifact:

```
pwsh -File .\AGBINV-Report-GovernanceRegister.ps1 `
     -InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv `
     -SharingCsv   C:\AgentGovernance\Exports\AgentBuilder-Sharing_<date>.csv `
     -OutputPath   C:\AgentGovernance\Exports
```

(Path A only: pass the admin-center CSV to `-InventoryCsv` and omit `-SharingCsv`; the register will carry inventory + owner, with sharing marked *"admin-center export — recipients not enumerated."*)

1. The generator writes `AgentBuilder-Governance-Register_<date>.csv` — one row per agent with:

agent name, owner, status/publish state, platform, created/modified, **sharing scope** (Org-wide / Specific / Only owner), **shared-with recipients** (semicolon-joined names, Path B), and blank review columns: `OwnerConfirmed`, `SharingApproved`, `Action` (Keep / Restrict sharing / Reassign owner / Block / Delete), `ReviewedBy`, `ReviewedOn`.

1. This CSV is the `Agent Builder Governance Register`. Store it in the R9 location.

Step 7 — Pilot: reconcile one slice before trusting the whole

1. Pick a small, known set — e.g. agents owned by your own pilot user or a single department.

2. Open Copilot Chat as (or with) an agent owner and note, for one agent, exactly who it is shared with (**Share** dialog → recipients).

1. Confirm that same agent's row in the register lists the identical recipients (Path B) or at least the correct owner and status (Path A). Resolve any mismatch now (usually a delegated-permission gap or a filter that dropped/added the wrong platform) — this is the trust gate.

Step 8 — Expand and record the governance decisions

1. Remove the pilot filter and export the full tenant set (Steps 3–6, or Step 5 for Path A).

2. Have the register **owner** review each row: confirm the owner, approve or challenge the sharing

scope (org-wide shares get first attention), and set `Action`.

1. Execute approved actions in the **Microsoft 365 admin center** (block, reassign owner) or by asking the agent owner to change sharing in Copilot; record the decision in the register.

1. Schedule the recurring export (see Best Practices) so the register stays current — the scheduled task is named `AGBINV – Monthly Agent Inventory Export`.

Testing & Validation

Each gate has an explicit **PASS / FAIL** criterion. Do not declare the register trustworthy until the gates that apply to your path pass. Path B adds the sharing-accuracy gates; Path A stops at inventory completeness.

Gate 1 — Preflight returns a clear, correct verdict (Path B)

- **Do:** Run `AGBINV-Connect-Preflight.ps1 -Environment <cloud>`.
- **PASS:** The script prints the signed-in account, its scopes (including `CopilotPackages.Read.All`), its roles, and a decoded HTTP result (`200`/`401`/`403`/`404`) with a named likely cause.
- **FAIL:** The script errors without a decoded verdict, or reports `200` but a later call fails — recheck R1/R2 and re-run.

Gate 2 — The API is reachable OR the fallback is confirmed

- **Do:** Read the preflight verdict.
- **PASS:** Either the preflight returns `200` (Path B available), **or** it returns `403`/`404` and you have confirmed the specific cause (R4 entitlement, R6 rollout, or Government cloud) and switched to **Path A**.
- **FAIL:** You are guessing which cause applies — go to Troubleshooting T2–T7 and distinguish them before proceeding.

Gate 3 — Inventory is scoped to Agent Builder only (no Copilot Studio)

- **Do:** Open `AgentBuilder-Inventory_<date>.csv` (Path B) or the filtered admin-center export (Path A).
- **PASS:** Every row's **Platform** is *Microsoft 365 Copilot Agent Builder*; no row shows *Copilot Studio* or a partner/Microsoft publisher. (Path B: every row's `ElementTypes` contains `declarativeAgent`.)
- **FAIL:** Any Copilot Studio or non-Agent-Builder row is present — the platform filter was not applied; re-run Step 3/Step 5 with the filter.

Gate 4 — Inventory is complete against the admin center

• **Do:** Compare the row count and agent names in the register against **Microsoft 365 admin center > Agents > All agents > Registry**, filtered to Platform = *Microsoft 365 Copilot Agent Builder*.

• **PASS:** The two lists match agent-for-agent (allowing for the minutes-level refresh lag between the portal and an export).

• **FAIL:** The register is missing agents present in the portal (or vice versa) — check paging in the export (the script must follow `@odata.nextLink`) and that no residual pilot filter is applied.

Gate 5 — Sharing recipients are accurate (Path B)

• **Do:** Pick two agents — one shared with **Specific users**, one shared with **Anyone in your organization**. For the first, open the owner's** Share** dialog and note the recipients. For the second, confirm it is a broad share.

• **PASS:** The **Specific users** agent's register row lists exactly the same users/groups (resolved to names); the **Anyone in your organization** agent is labeled **Org-wide shared** (not an empty recipient list).

• **FAIL:** Recipients differ, are missing, or an org-wide share shows blank — most often a delegated scope/consent gap or an unresolved object ID (see Troubleshooting T5).

Gate 6 — Group recipients resolve to real groups (Path B)

• **Do:** For any agent shared to a group, confirm the register's `RecipientName` matches a real Entra group.

• **PASS:** Each group `resourceId` resolves to a group display name; no rows show a raw GUID as the name.

• **FAIL:** Raw GUIDs remain — the resolver lacked directory read or the object is a different type; verify the account can read `GET /groups/{id}` and `GET /users/{id}`.

Gate 7 — Owners are populated and ownerless agents are flagged

• **Do:** Scan the register's **Owner** column.

• **PASS:** Every agent has an owner, or agents with no owner are explicitly flagged **(ownerless)** — matching the admin center's *Agents without owners* count.

• **FAIL:** Owner is blank without the ownerless flag — reconcile against the admin center's ownerless filter.

Gate 8 — The register is safe-to-store and reviewable

• **Do:** Confirm the register lives in the R9 location and carries the review columns.

• **PASS:** `AgentBuilder-Governance-Register_<date>.csv` is in `C:\AgentGovernance\Exports` (or the

governed SharePoint location), access is restricted to the governance team, and the `OwnerConfirmed` / `SharingApproved` / `Action` columns are present and empty, ready for review.

• **FAIL:** The file is in an uncontrolled location or lacks the review columns — re-run Step 6 and move it into the controlled store.

Gate 9 — The recurring export runs unattended (after Best Practices)

• **Do:** After scheduling, trigger `AGBINV – Monthly Agent Inventory Export` once manually.

• **PASS:** A fresh dated register appears in `C:\AgentGovernance\Exports` with no interactive sign-in.

• **FAIL:** The task prompts for credentials or errors — app-only auth (R3) is not configured; fix the certificate/consent and re-test.

Gate 10 — Government cloud behaves as documented (if applicable)

• **Do:** In a GCC High/DoD tenant, run the preflight with `-Environment USGov/USGovDoD`.

• **PASS:** The preflight reports the Package API is unavailable in that cloud and directs you to Path A; the admin-center export produces the inventory.

• **FAIL:** You assumed Path B would work in Government and treated its failure as a permissions bug — re-read R7 and use Path A. **Verify for GCC High.**

Troubleshooting

This is the **blocked/failed-request decision tree** for the customer's exact situation. Start at T1, read the HTTP status, and follow the branch. The goal is to tell the causes apart — permission vs role vs Agent 365 entitlement vs feature-not-enabled vs undocumented-use vs a genuine product limitation — so you fix the real one instead of guessing.

T1 — First, capture the exact status code and the endpoint

The single most useful action is to see the raw HTTP status. Run the preflight (`AGBINV-Connect-Preflight.ps1`) or call the endpoint directly:

```
Connect-MgGraph -Environment Global -Scopes CopilotPackages.Read.All
Invoke-MgGraphRequest -Method GET -Uri 'https://graph.microsoft.com/v1.0/copilot/admin/catalog/packages?$t
```

• Confirm the **URL/namespace** is `/copilot/admin/catalog/packages` (correct) — not `/admin/...` or `/solutions/...`.

• Note the status: `401` → T2. `403` → T3–T6. `404` → T7. `200` **but wrong/empty data** → T8–T9.

T2 — `401 Unauthorized`: it is the token, not permissions

• **Symptom:** `401` before any data check.

• **Cause:** No token, an expired token, or a token minted for the wrong cloud (e.g. a Commercial token against `graph.microsoft.us`).

• **Fix:** Re-run `Connect-MgGraph` with the correct `-Environment` for your tenant (R7). Confirm the token audience is `https://graph.microsoft.com` (or the `.us` host for Government). Do not touch roles or scopes yet — a `401` never means the role is wrong.

T3 — `403` decision: is it the SCOPE, the ROLE, the ENTITLEMENT, or the ROLLOUT?

A `403` means "authenticated but not allowed," and four different causes produce it. Work them in this order — the preflight prints them in the same priority:

1. **Scope / consent (most common — this is usually the customer's cause).** Decode the token at `jwt.ms` and check the `scp` claim for `CopilotPackages.Read.All`. If it is absent, the scope was not requested or admin consent was never granted. → T4.

1. **Admin role.** If the scope is present, confirm the signed-in identity holds a qualifying role (**AI Administrator**, Global Administrator, Cloud Application Administrator, Application Administrator, or Copilot Administrator). A correct scope with an unqualified role still returns `403`. → T5.

1. **Agent 365 entitlement.** If both scope and role are correct and you *still* get `403`, the current docs' **Microsoft Agent 365** requirement for this API (R4) is the leading suspect. → T6.

1. **Feature not rolled out.** A tenant that has not received the API can also surface `403` at the backend before permission evaluation. → T7 (usually presents as `404`, sometimes `403`).

T4 — `403` caused by missing scope or consent

• **Symptom:** `scp` claim lacks `CopilotPackages.Read.All`.

• **Cause:** The delegated app never requested the scope, or a Global Administrator never consented to it for the tenant, or (application flow) the app permission was added but consent not granted.

• **Fix:**

• **Delegated:** run `Connect-MgGraph -Scopes CopilotPackages.Read.All`; on the consent prompt, a Global Administrator selects **Consent on behalf of your organization**. Re-acquire the token and retry.

• **Application:** in the Entra app `AGBINV – Agent Governance – Graph Reader` > **API permissions** *add* `CopilotPackages.Read.All` *(Application) >* ***Grant admin consent for <tenant>*** *(state must read *Granted*). Re-run.*

• **Confirm:** the `scp`/`roles` claim now contains the scope and the call returns `200` (or advances to

the next 403 cause).

T5 — 403 caused by an insufficient admin role

- **Symptom:** Scope is present, but 403 persists.
- **Cause:** The account lacks a qualifying admin role, or the role was assigned after the current token

was issued.

- **Fix:** Assign **AI Administrator** (least-privileged practical choice) in **Entra admin center** >

Roles & administrators > **AI Administrator** > **Add assignments**; then **sign out and back in** so the new role is reflected in a fresh token. Retry.

- **Confirm:** (`Get-MgContext`) shows the account, and the call returns 200 (or advances to T6).

T6 — 403 caused by the Microsoft Agent 365 entitlement (verify — you ruled this out)

- **Symptom:** Scope **and** role are both correct, the API is live for the tenant (other Copilot admin

APIs work), yet packages still returns 403.

- **Cause:** The **current** Microsoft Learn pages state the Package Management API **requires a

Microsoft Agent 365 license**. If the entitlement is enforced and absent, this is the residual 403.

- **You told us Agent 365 is ruled out.** Take that at face value for your decision, but verify honestly:

check **Microsoft 365 admin center** > **Billing** > **Licenses/Subscriptions** for an Agent 365 / agent-management subscription. If it is genuinely absent **and** scope+role are correct, then the supported programmatic path is currently gated for your tenant.

- **Fix / path forward:** Use **Path A — the Microsoft 365 admin center Agent Registry Export to CSV**

(Implementation Step 5). It produces the Agent Builder inventory **without** Agent 365. The granular `allowedUsersAndGroups` sharing detail from Path B becomes available only if/when the entitlement is in place. *(Verify against current Microsoft documentation — Microsoft may change this entitlement as the API matures; re-test Path B after any change.)*

T7 — 404 (or backend 403): the API is not rolled out to your tenant/cloud

- **Symptom:** `404 Not Found` on a correctly-formed `/copilot/admin/catalog/packages` URL with a valid

token and qualifying role; or a 403 that persists after T4–T6 are cleanly ruled out.

- **Cause:** The Package Management API released **tenant-by-tenant**; your tenant has not received it

yet. Or you are in a **Government cloud** (GCC High L4 / DoD L5) where the API is **not available at all** (R7).

- **Fix:** There is no self-service switch to force earlier availability. Use **Path A** now and re-test

Path B after Microsoft confirms general availability for your cloud (track **Message Center MC1173195** and Microsoft Learn). In Government clouds, Path A is the standing method — do not wait for Path B. **Verify**

for GCC High.

- **Confirm:** re-running the preflight later returns `200` once the tenant is enabled.

T8 — `200` but the result is empty or missing agents

- **Symptom:** The call succeeds but returns few or no Agent Builder agents.
- **Causes and fixes:**
- **Legitimately empty** — no Agent Builder agents exist yet (R5). Confirm in the admin center Registry

under *Shared by creator* / Platform = *Microsoft 365 Copilot Agent Builder*.

- **Over-filtered** — the `platform`/`elementTypes` filter excluded valid agents (e.g. a casing or

value mismatch). Re-list without the filter, inspect the `platform` values actually returned, then reapply the exact string `Microsoft 365 Copilot Agent Builder`.

- **Paging dropped** — the script did not follow `@odata.nextLink`, so only the first page returned.

Confirm the export loops until `nextLink` is null (Gate 4).

T9 — `200` on the list, but no per-user sharing appears

- **Symptom:** The inventory lists agents, but the sharing export shows empty recipients.
- **Causes and fixes:**
- **You only called the list endpoint.** The list `copilotPackage` object exposes `availableTo` /

`deployedTo` as `all`/`some`/`none` — **not** the recipients. You must call `GET .../packages/{id}` per agent to read `allowedUsersAndGroups` (Implementation Step 4). This is the #1 reason "sharing is missing."

- **Org-wide share.** An agent shared with **Anyone in your organization** has `availableTo = all` and

may carry no explicit per-user entries — that is correct; the register labels it **Org-wide shared**, not blank.

- **Unresolved object IDs.** Recipients show as raw GUIDs when the account cannot read `GET /users/{id}`

/ `GET /groups/{id}`. Grant the resolver directory read (a Global/Directory Reader-level scope) and re-run (Gate 6).

T10 — Distinguishing an undocumented/wrong endpoint from a real limitation

- **If you used a different path** (e.g. `/beta/admin/...` or a `/solutions/...` guess), you will get

`404` because the resource does not exist — that is a **wrong-endpoint** error, not a limitation. The correct, documented namespace is `/copilot/admin/catalog/packages` (list) and `/{id}` (detail).

- **A genuine product limitation** (as opposed to a fixable block) is narrow: the API returns the

allowed audience (`allowedUsersAndGroups`), **not** a runtime log of who actually opened or ran the agent. If your requirement is *usage* rather than *who it is shared with*, that data is not in this API — source it from Copilot usage reporting or the unified audit log instead. Everything else the customer asked for (inventory + sharing recipients, filtered to Agent Builder) **is** supported; the failures are blocks to clear (T4–T7), not dead ends.

Best Practices

Stay read-only for governance reporting

Inventory and sharing export never need to change an agent. Use `CopilotPackages.Read.All`, not the `ReadWrite` scope. Reserve any change action (block, reassign owner) for the Microsoft 365 admin center, where it is audited, rather than baking writes into the export.

Prefer `v1.0` over `beta`, and record the version you used

Both `v1.0` and `beta` expose `/copilot/admin/catalog/packages`. Use `v1.0` in the scripts and the scheduled task for production stability; keep `beta` only for testing a newly documented property. Record the API version and the run date in the register header so a later reader knows exactly what produced it — this API is still settling, so provenance matters.

Make the export recurring — schedule the monthly run

Agent Builder agents are created and shared continuously, so a one-time inventory is stale within weeks. Schedule `AGBINV – Monthly Agent Inventory Export` to run the inventory → sharing → register sequence app-only (R3), writing a dated register each month into `C:\AgentGovernance\Exports`. Keep the last 12 months so you can diff month-over-month and see new org-wide shares appear.

- Windows Task Scheduler: create task `AGBINV – Monthly Agent Inventory Export`, run whether or not

the user is logged on, action = `pwsh.exe -File AGBINV-Report-GovernanceRegister.ps1 ...` wrapped in a runner that first calls the inventory and sharing scripts with the app-only certificate.

Triage by sharing scope, not alphabetically

When reviewing the register, sort by **sharing scope** and work the riskiest first:

1. **Org-wide shared** (`availableTo = all` / "Anyone in your organization") — broadest exposure; confirm

the agent and its knowledge sources are appropriate for everyone.

1. **Shared to large groups** — a single group share can reach hundreds of people.
2. **Ownerless agents** — no accountable owner; decide reassign vs block vs delete.
3. **Specific-user shares** — lowest exposure; spot-check.

Keep the register access-controlled — it names people

The register lists users and groups by name. Treat it as sensitive: store it in a location restricted to the governance team (R9), do not email it around, and apply your normal retention policy. Never publish it to a broadly-shared site.

Distinguish "can reach" from "has used"

The register records the **allowed audience** (`allowedUsersAndGroups`), not runtime usage. Do not present it as a usage report. If leadership needs adoption/usage, source that separately from Copilot usage reporting or the audit log — and label it as a different metric.

Keep Agent Builder and Copilot Studio governance separate

This register is deliberately scoped to Agent Builder. Your Copilot Studio approval gates govern the Copilot Studio population; do not merge the two into one list or you lose the meaning of each control. Keep the platform filter in place on every run.

Least privilege and time-bound admin

Run governance with **AI Administrator** (read-oriented) rather than Global Administrator, and make the role assignment **eligible/time-bound** via Privileged Identity Management. The unattended app (`AGBINV – Agent Governance – Graph Reader`) should hold only `CopilotPackages.Read.All` plus the minimal directory read needed to resolve names — nothing more.

Government cloud: use the supported path and re-test the rest

In GCC High/DoD, the Package Management API is not available today — build the register from the admin center export (Path A) and re-test Path B only after Microsoft documents its availability for your cloud. Verify Agent Builder availability itself in your Government cloud. **Verify for GCC High** rather than assuming Commercial behavior.

Deliverables Summary

What you receive

• **Solution Overview** — what Agent Builder agents are, where they surface for admins, the

beta-endpoint reality (the direct answer on `GET /copilot/admin/catalog/packages/{id}`), and the two-path export approach.

• **Requirements** — nine requirements (roles, the exact Graph scope `CopilotPackages.Read.All`, the

Agent 365 entitlement reality, tenant rollout, cloud selection, storage) each with Description / Why / Verify / Implement-if-missing, plus the Commercial-vs-Government endpoint table.

• **Implementation** — the ordered build: choose cloud/path, preflight and decode the failure, list

agents filtered to Agent Builder, resolve sharing to users/groups, build the register, pilot, expand.

• **Testing** — ten PASS/FAIL gates from "preflight returns a clear verdict" to "Government behaves as

documented."

• **Troubleshooting** — the blocked/failed-request decision tree: permission vs role vs Agent 365 vs feature-not-enabled vs undocumented vs real limitation.

• **Best Practices** — read-only posture, `v1.0` over `beta`, the recurring monthly export, triage by sharing scope, least privilege, and Government-cloud guidance.

• **Executive Summary** — the one-page brief for the governance sponsor.

• **Category global requirements** — the Copilot-category common prerequisites.

The five PowerShell scripts

All accept `-Environment Global|USGov|USGovDoD` and are **read-only** against Microsoft Graph. Each header carries Purpose / How to run / Expected results.

Script	What it does
`AGBINV-Connect-Preflight.ps1`	Signs in, reports scopes + roles, calls the packages endpoint, and **decodes** the `401`/`403`/`404` into a named cause — the "why is my request blocked" answer.
`AGBINV-Export-AgentInventory.ps1`	Lists Copilot packages **filtered to the Agent Builder platform +** `declarativeAgent` and writes the inventory CSV.
`AGBINV-Export-AgentSharing.ps1`	For each agent, calls the **per-id detail** endpoint, reads `allowedUsersAndGroups`, and resolves users/groups to names → sharing CSV.
`AGBINV-Filter-AgentBuilderOnly.ps1`	Reduces any packages export to **Agent Builder only**, excluding Copilot Studio / Microsoft / partner agents.
`AGBINV-Report-GovernanceRegister.ps1`	Merges inventory + sharing into the `Agent Builder Governance Register` with review columns.

The artifact you maintain

• `Agent Builder Governance Register` (`AgentBuilder-Governance-Register_<date>.csv`) — one row per Agent Builder agent: owner, status, platform, dates, **sharing scope**, **shared-with recipients**, and review columns (OwnerConfirmed / SharingApproved / Action / ReviewedBy / ReviewedOn). Stored in the controlled export location and refreshed by the monthly scheduled task.

Format

Delivered as a Word document and PDF, plus all PowerShell scripts in a single ZIP — yours to keep and reuse across unlimited runs.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AGBINV-Connect-Preflight.ps1

Purpose: Answers "why is my /copilot/admin/catalog/packages request blocked or failed." Signs in, reports the scopes and directory roles on the current context, then calls the packages endpoint and decodes the HTTP result into a specific, actionable cause (missing scope/consent, insufficient role, Agent 365 entitlement, or tenant not rolled out / cloud not supported). Read-only; changes nothing. HOW TO RUN pwsh -File .\AGBINV-Connect-Preflight.ps1 -Environment Global Government clouds: -Environment USGov (GCC High) or -Environment USGovDoD (DoD) Required delegated scope: CopilotPackages.Read.All (least privilege; read-only) Qualifying roles: AI Administrator (preferred), Global Administrator, Cloud Application Administrator, Application Administrator, or Copilot Administrator. EXPECTED RESULTS A printed verdict: signed-in account, scopes, roles, and a decoded HTTP status with the most likely cause. 200 = API live and authorized (use Path B). 401 = token. 403 = scope/role/ entitlement. 404 = not rolled out / cloud unsupported (use Path A — admin center export). .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. The Package Management API released recently and is documented in both v1.0 and beta; this script uses v1.0 and treats availability as tenant-dependent. Verify preview-volatile behavior against current Microsoft docs. >

How to run: pwsh -File .\AGBINV-Connect-Preflight.ps1 -Environment Global Government clouds: -Environment USGov (GCC High) or -Environment USGovDoD (DoD) Required delegated scope: CopilotPackages.Read.All (least privilege; read-only) Qualifying roles: AI Administrator (preferred), Global Administrator, Cloud Application Administrator, Application Administrator, or Copilot Administrator. EXPECTED RESULTS A printed verdict: signed-in account, scopes, roles, and a decoded HTTP status with the most likely cause. 200 = API live and authorized (use Path B). 401 = token. 403 = scope/role/ entitlement. 404 = not rolled out / cloud unsupported (use Path A — admin center export). .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. The Package Management API released recently and is documented in both v1.0 and beta; this script uses v1.0 and treats availability as tenant-dependent. Verify preview-volatile behavior against current Microsoft docs. >

AGBINV-Export-AgentInventory.ps1

Purpose: Lists Copilot packages via GET /copilot/admin/catalog/packages, filtered to the Agent Builder platform, and keeps only declarative agents. Writes one row per agent (name, owner-adjacent fields, status, dates, publish state). This is the inventory backbone of the governance register. Read-only; changes nothing. HOW TO RUN pwsh -File .\AGBINV-Export-AgentInventory.ps1 -Environment Global ` -OutputPath C:\AgentGovernance\Exports Government clouds: -Environment USGov | USGovDoD (NOTE: the Package API is Global-only today; in Government clouds use the admin center export — Path A. The script reports if unavailable.) Required scope: CopilotPackages.Read.All EXPECTED RESULTS AgentBuilder-Inventory_<yyyyMMdd-HHmm>.csv in -OutputPath — one row per Agent Builder agent with Id, DisplayName, Type, Publisher, Platform, ElementTypes, IsBlocked, AvailableTo, DeployedTo, LastModifiedDateTime, Version. AvailableTo/DeployedTo are all/some/none summaries; the per-recipient sharing comes from AGBINV-Export-AgentSharing.ps1. .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. Uses v1.0; falls back to beta only if v1.0 returns 404 for the resource. Preview-volatile — verify against current docs. >

How to run: pwsh -File .\AGBINV-Export-AgentInventory.ps1 -Environment Global ` -OutputPath C:\AgentGovernance\Exports Government clouds: -Environment USGov | USGovDoD (NOTE: the Package API is Global-only today; in Government clouds use the admin center export — Path A. The script reports if unavailable.) Required scope: CopilotPackages.Read.All EXPECTED RESULTS AgentBuilder-Inventory_<yyyyMMdd-HHmm>.csv in -OutputPath — one row per Agent Builder agent with Id, DisplayName, Type, Publisher, Platform, ElementTypes, IsBlocked, AvailableTo, DeployedTo, LastModifiedDateTime, Version. AvailableTo/DeployedTo are all/some/none summaries; the per-recipient sharing comes from AGBINV-Export-AgentSharing.ps1. .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. Uses v1.0; falls back to beta only if v1.0 returns 404 for the resource. Preview-volatile — verify against current docs. >

AGBINV-Export-AgentSharing.ps1

Purpose: The sharing exporter. For every agent in the inventory CSV, calls GET /copilot/admin/catalog/packages/{id} and reads the copilotPackageDetail property allowedUsersAndGroups (each entry is { resourceId, resourceType = user|group }) — the granular "shared with whom" data. Resolves each id to a display name / UPN via /users/{id} or /groups/{id}. Also captures availableTo/deployedTo and sensitivity. Read-only; changes nothing. HOW TO RUN pwsh -File .\AGBINV-Export-AgentSharing.ps1 -Environment Global `-InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv` -OutputPath C:\AgentGovernance\Exports Required scopes: CopilotPackages.Read.All (and directory read to resolve names — User.Read.All / Group.Read.All or Directory.Read.All) EXPECTED RESULTS AgentBuilder-Sharing_<yyyyMMdd-HHmm>.csv — one row per agent x recipient, with AgentId, AgentName, SharingScope (OrgWide/Specific/OnlyOwner), RecipientType, RecipientId, RecipientName, RecipientUpnOrMail. Org-wide shares (availableTo=all) are labeled OrgWide rather than shown as an empty recipient list. .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. The list endpoint only exposes all/some/none; the per-id detail exposes allowedUsersAndGroups — this script uses the per-id detail. Preview-volatile — verify property names against current docs. >

How to run: pwsh -File .\AGBINV-Export-AgentSharing.ps1 -Environment Global `-InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv` -OutputPath C:\AgentGovernance\Exports Required scopes: CopilotPackages.Read.All (and directory read to resolve names — User.Read.All / Group.Read.All or Directory.Read.All) EXPECTED RESULTS AgentBuilder-Sharing_<yyyyMMdd-HHmm>.csv — one row per agent x recipient, with AgentId, AgentName, SharingScope (OrgWide/Specific/OnlyOwner), RecipientType, RecipientId, RecipientName, RecipientUpnOrMail. Org-wide shares (availableTo=all) are labeled OrgWide rather than shown as an empty recipient list. .NOTES Copilot Solutions — the repeatable way. Read-only against Microsoft Graph. The list endpoint only exposes all/some/none; the per-id detail exposes allowedUsersAndGroups — this script uses the per-id detail. Preview-volatile — verify property names against current docs. >

AGBINV-Filter-AgentBuilderOnly.ps1

Purpose: A standalone filter. Given a CSV that contains a mixed packages/agents export (from Graph or the admin center Registry export), keeps only rows whose Platform is the Agent Builder platform and whose ElementTypes include a declarative agent — so the governance register never mixes in Copilot Studio agents (which are governed separately by your Copilot Studio approval gates). Read-only; operates on files, never calls Graph. HOW TO RUN pwsh -File .\AGBINV-Filter-AgentBuilderOnly.ps1 `-InputCsv C:\AgentGovernance\Exports\all-agents-export.csv` -OutputPath C:\AgentGovernance\Exports Optional: -PlatformColumn / -ElementTypesColumn if your source uses

different headers (the admin center export labels the column 'Platform'). EXPECTED RESULTS AgentBuilder-Filtered_<yyyyMMdd-HHmm>.csv containing only Agent Builder declarative agents, plus a console summary of how many rows were kept vs excluded and which platforms were dropped. .NOTES Copilot Solutions — the repeatable way. File-only; no Graph calls, no changes to Intune/Entra. >

How to run: pwsh -File .\AGBINV-Filter-AgentBuilderOnly.ps1 `-InputCsv C:\AgentGovernance\Exports\all-agents-export.csv` -OutputPath C:\AgentGovernance\Exports Optional: -PlatformColumn / -ElementTypesColumn if your source uses different headers (the admin center export labels the column 'Platform'). EXPECTED RESULTS AgentBuilder-Filtered_<yyyyMMdd-HHmm>.csv containing only Agent Builder declarative agents, plus a console summary of how many rows were kept vs excluded and which platforms were dropped. .NOTES Copilot Solutions — the repeatable way. File-only; no Graph calls, no changes to Intune/Entra. >

AGBINV-Report-GovernanceRegister.ps1

Purpose: Builds the review artifact. Joins the inventory CSV (one row per agent) with the sharing CSV (one row per agent x recipient) into a single register: one row per agent, sharing recipients collapsed to a readable list, sharing scope surfaced, and blank governance-review columns added (OwnerConfirmed / SharingApproved / Action / ReviewedBy / ReviewedOn). File-only; no Graph. HOW TO RUN Path B (with granular sharing): pwsh -File .\AGBINV-Report-GovernanceRegister.ps1 `-InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv` -SharingCsv C:\AgentGovernance\Exports\AgentBuilder-Sharing_<date>.csv `-OutputPath C:\AgentGovernance\Exports Path A (admin center export, no per-recipient sharing): pwsh -File .\AGBINV-Report-GovernanceRegister.ps1` -InventoryCsv C:\AgentGovernance\Exports\admin-center-export.csv -OutputPath C:\AgentGovernance\Exports EXPECTED RESULTS AgentBuilder-Governance-Register_<yyyyMMdd-HHmm>.csv — the register. If -SharingCsv is omitted, the SharedWith column reads "admin-center export - recipients not enumerated (use Path B)". .NOTES Copilot Solutions — the repeatable way. File-only; no Graph calls. Store the register in an access-controlled location — it names users and groups. >

How to run: Path B (with granular sharing): pwsh -File .\AGBINV-Report-GovernanceRegister.ps1 `-InventoryCsv C:\AgentGovernance\Exports\AgentBuilder-Inventory_<date>.csv` -SharingCsv C:\AgentGovernance\Exports\AgentBuilder-Sharing_<date>.csv `-OutputPath C:\AgentGovernance\Exports Path A (admin center export, no per-recipient sharing): pwsh -File .\AGBINV-Report-GovernanceRegister.ps1` -InventoryCsv C:\AgentGovernance\Exports\admin-center-export.csv -OutputPath C:\AgentGovernance\Exports EXPECTED RESULTS AgentBuilder-Governance-Register_<yyyyMMdd-HHmm>.csv — the register. If -SharingCsv is omitted, the SharedWith column reads "admin-center export - recipients not enumerated (use Path B)". .NOTES Copilot Solutions — the repeatable way. File-only; no Graph calls. Store the register in an access-controlled location — it names users and groups. >